

07

Módulo 07 - Tipos de aprendizagem - aprendizagem não supervisionada

Fundamentos de Inteligência Artificial (IA)

Resumo do Módulo

- ◆ Como Encontrar Padrões em Dados Não Etiquetados.
- ◆ Algoritmos de Clustering (ex: K-means).
- ◆ Algoritmos de Redução de Dimensionalidade (ex: PCA)
- ◆ Lab

Machine Learning

◆ Supervisionada (Supervised):

- Regressão (Regression): ~~Linear Simples~~ • ~~Linear Múltipla~~ • ~~Polinomial~~ • ~~Lasso~~ • ~~Ridge~~
- Classificação (Classification): ~~Regressão Logística~~ • ~~KNN~~ • ~~SVM~~ • ~~Kernel SVM~~ • ~~Naïve Bayes~~
- Séries Temporais (Time Series): ~~ARIMA~~ • ~~SARIMA~~ • ~~LSTM~~ • ~~Prophet~~ • ~~Suavização Exponencial~~
- Métodos de Conjunto (Ensemble Methods): ~~Árvore de Decisão~~ • ~~Random Forest~~ • ~~XGBoost~~ • ~~CatBoost~~
- Aprendizagem Profunda (Deep Learning): ~~Redes Neurais Artificiais~~ • ~~Convolucionais~~ • ~~Recorrentes~~ • ~~Transformers~~ + ~~Atenção~~

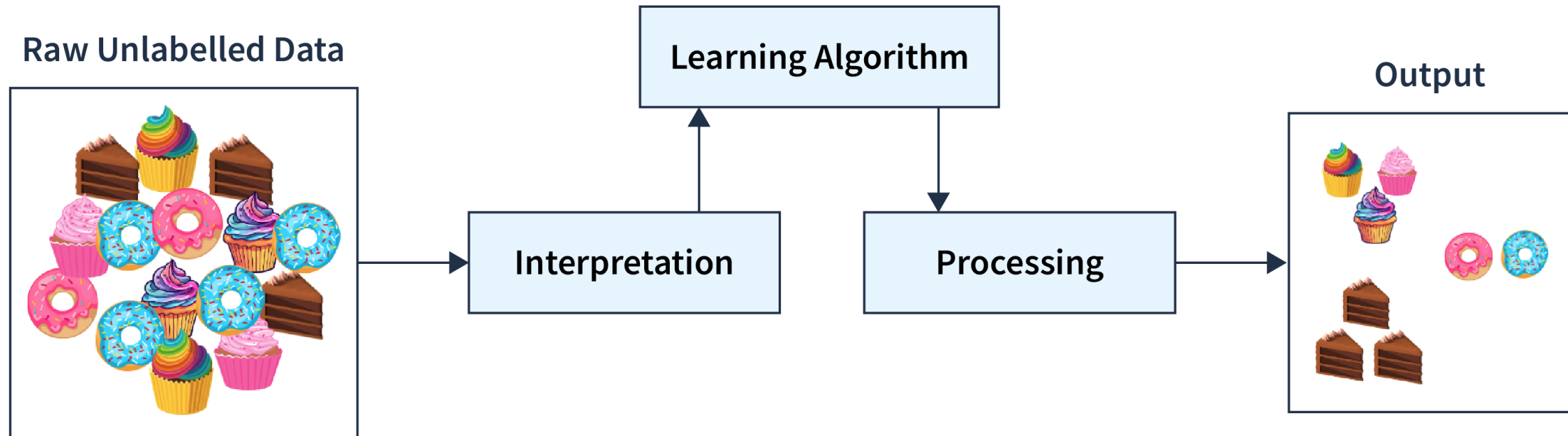
◆ Não Supervisionada (Unsupervised):

- Clustering (Agrupamento): ~~K-means~~ • ~~Hierárquico~~ • ~~GMM~~
- Redução de Dimensionalidade (Dimensionality Reduction): ~~PCA~~ • ~~Kernel PCA~~ • ~~LDA~~

◆ Reforço (Reinforcement):

- Algoritmos de Reforço (Reinforcement Algorithms): ~~Limite de Confiança Superior~~ • ~~Amostragem de Thompson~~ • ~~Q-Learning~~ • ~~Deep Q-Learning~~ • ~~PPO~~
- Algoritmos Genéticos (Genetic Algorithms): ~~Algoritmo Genético Clássico~~ • ~~NSGA-II~~ • ~~Algoritmos Meméticos~~

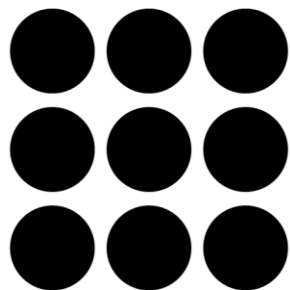
Aprendizagem não supervisionada (Unsupervised Machine Learning)



Como Encontrar Padrões em Dados Não Etiquetados

Através da identificação de **similaridades** e **diferenças** entre os dados!

Como o ser humano!



Princípios de Gestalt

O quê: Leis da psicologia que explicam como a mente organiza informação visual.

Quando: Início do século XX.

Onde: Alemanha.

Quem: Psicólogos como Wertheimer, Köhler e Koffka.

Objetivo: Entender como percebemos unidades significativas a partir de elementos visuais separados.



Comparar algoritmos de Clustering (Agrupamento)

Algoritmo	Prós	Contras
K-means	Simples e rápido; bom para grandes conjuntos de dados	Necessita definir o número de clusters; sensível a outliers
Clustering Hierárquico	Não precisa definir o número de clusters a priori; fácil de visualizar	Ineficiente em grandes conjuntos de dados; difícil ajustar o corte do dendrograma
GMM (Gaussian Mixture Model)	Captura formas de cluster mais complexas; permite sobreposição de clusters	Pode ser sensível a inicializações; requer ajuste de múltiplos parâmetros

K-means

Agrupar pontos de dados semelhantes, minimizando a variação dentro dos grupos e maximizando a diferença entre grupos.

- **Iterativo**: Calcula os centróides e ajusta os grupos até a convergência

```
# pip install scikit-learn
```

```
from sklearn.cluster import KMeans
import numpy as np
```

```
# Dados de exemplo
```

```
X = np.array([[1, 2], [1, 4], [1, 0], #...
              [4, 2], [4, 4], [4, 0]])
```

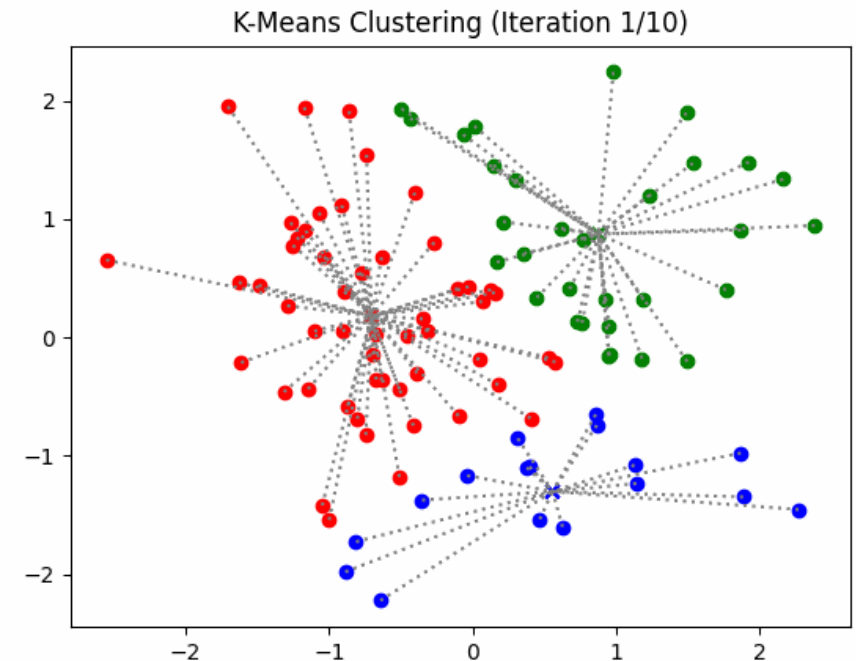
```
# Criar o modelo K-Means com K=3 (3 clusters)
```

```
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X)
```

```
# Fazer previsões
```

```
labels = kmeans.predict([[0, 0], [4, 4]])
```

```
print(labels) # Rótulos dos clusters para os novos pontos
```



Clustering Hierárquico (Hierarchical Clustering)

Organiza os dados em uma hierarquia de grupos.

```
# pip install scikit-learn
```

```
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
import numpy as np
```

```
# Dados de exemplo
```

```
X = np.array([[1,1], [3,2], [9,1], [3,7],
              [7,2], [9,7], [4,8], [8,3],[1,4]])
```

```
# Aplicar Clustering Hierárquico Aglomerativo
```

```
clustering = AgglomerativeClustering(n_clusters=2)
```

```
clustering.fit(X)
```

```
print(clustering.labels_) # Rótulos dos clusters
```

```
# Criar o linkage matrix e plot do dendrograma
```

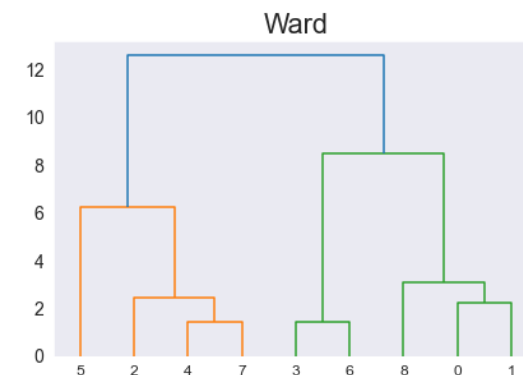
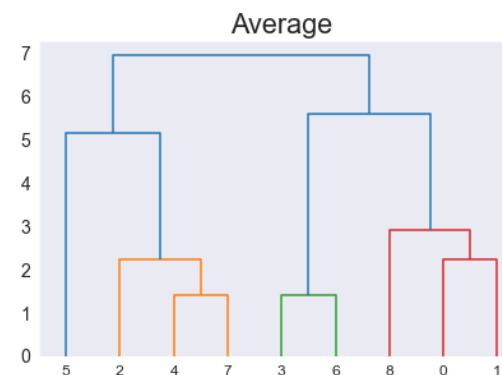
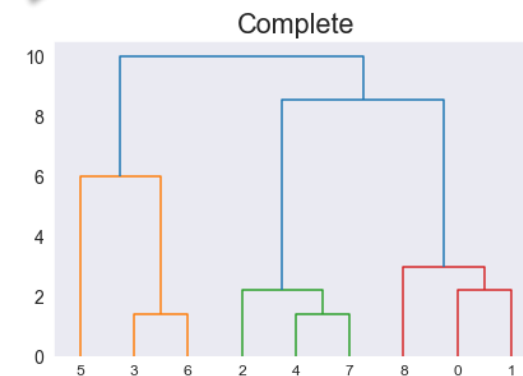
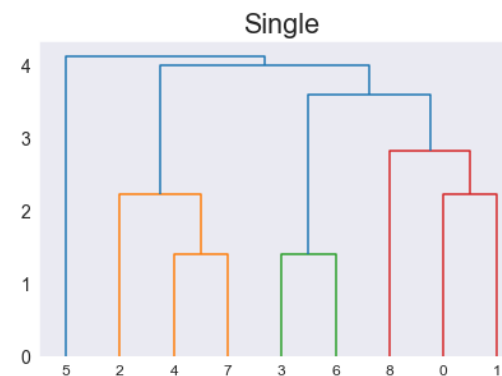
```
Z = linkage(X, 'ward') # 'ward': método de agrupamento
```

```
plt.figure(figsize=(8, 4))
```

```
dendrogram(Z)
```

```
plt.show()
```

4 Dendrograms



GMM (Gaussian Mixture Model)

Algoritmo de **Agrupamento Probabilístico**: Cada cluster é modelado como uma distribuição normal, permitindo formas elípticas e sobreposição entre clusters

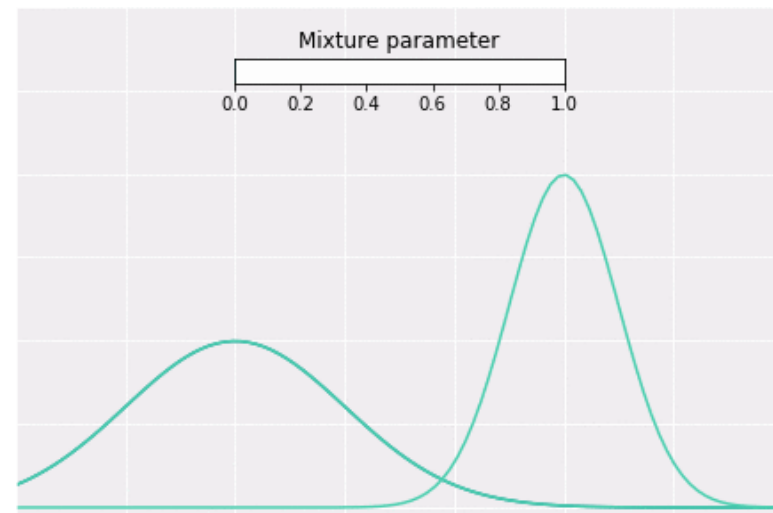
```
# pip install scikit-learn

from sklearn.mixture import GaussianMixture
import numpy as np

# Dados de exemplo
X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])

# Criar e ajustar o modelo GMM com 2 componentes (clusters)
gmm = GaussianMixture(n_components=2)
gmm.fit(X)

# Fazer previsões
labels = gmm.predict([[0, 0], [12, 3]])
print(labels) # Rótulos dos clusters para os novos pontos
```



- ◆ Lab 07 - Tipos de aprendizagem - aprendizagem não supervisionada
 - 7.1. - Clustering (Agrupamento) — Descoberta de Perfis Operacionais de Voo

Comparar algoritmos de Redução de Dimensionalidade (Dimensionality Reduct.)

Algoritmo	Prós	Contras
PCA (Principal Component Analysis)	Reduz a dimensionalidade mantendo variância; fácil de interpretar	Assume linearidade nos dados; pode não funcionar bem para padrões não-lineares
Kernel PCA	Captura padrões não-lineares; útil para dados complexos	Alto custo computacional; requer seleção do kernel adequado
LDA (Linear Discriminant Analysis)	Reduz a dimensionalidade maximizando a separabilidade entre classes	Funciona bem apenas para dados rotulados; limitado a relações lineares

PCA (Análise de Componentes Principais)

- ♦ **Reduz** o número de variáveis (features) de um conjunto de dados mantendo a maior parte da variabilidade original.

```
# pip install scikit-learn
```

```
from sklearn.decomposition import PCA  
import numpy as np
```

```
# Dados de exemplo
```

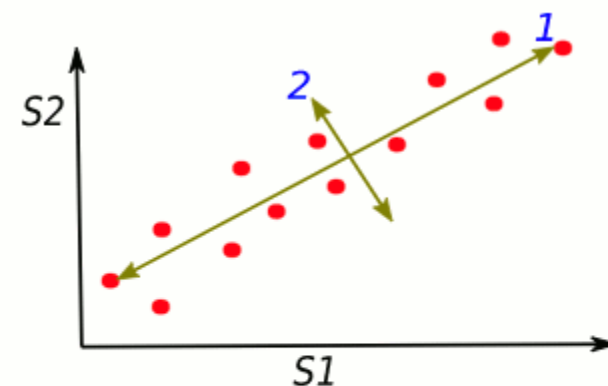
```
X = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0],  
[2.3, 2.7]])
```

```
# Criar e ajustar o modelo PCA com 1 componente principal
```

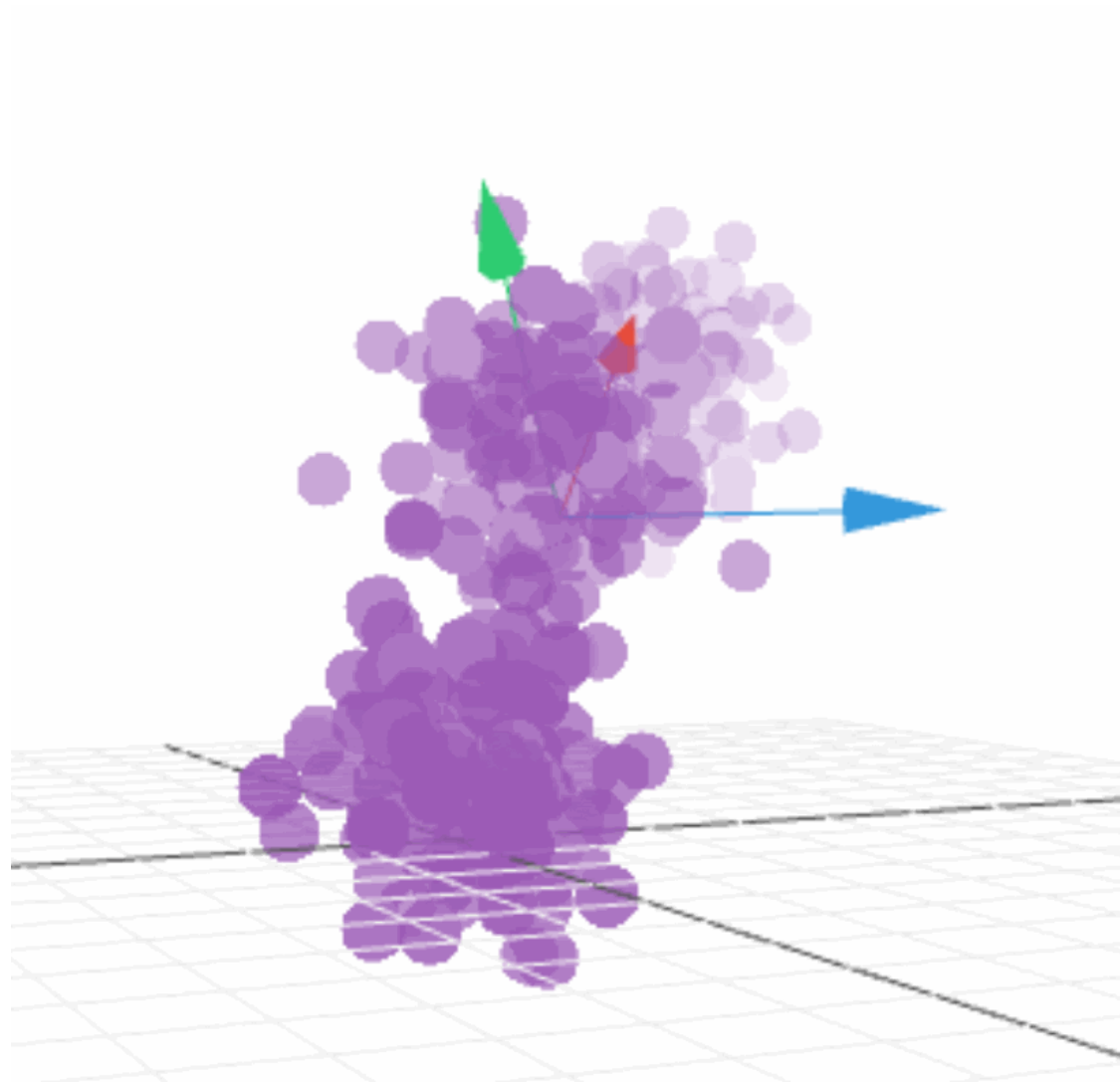
```
pca = PCA(n_components=1)  
X_reduced = pca.fit_transform(X)
```

```
# Componentes principais e variância explicada
```

```
print("Componentes Principais:\n", pca.components_)  
print("Variância Explicada:", pca.explained_variance_ratio_)
```



PCA em 3D



Kernel PCA (Análise de Componentes Principais com Kernel)

Extensão do PCA Tradicional

- ◆ Utiliza a técnica de kernel para lidar com dados não-lineares

```
# pip install scikit-learn
```

```
from sklearn.decomposition import KernelPCA  
import numpy as np
```

```
# Dados de exemplo
```

```
X = np.array([[0, 0], [1, 0], [0, 1], [1, 1], [0.5, 0.5]])
```

```
# Criar e ajustar o modelo Kernel PCA com o
```

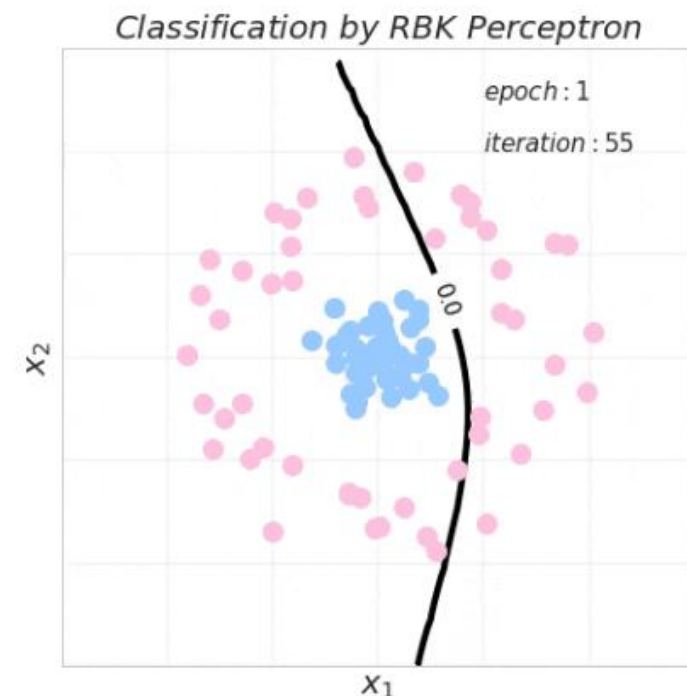
```
# kernel RBF (Radial Basis Kernel)
```

```
kpca = KernelPCA(n_components=2, kernel='rbf')
```

```
X_kpca = kpca.fit_transform(X)
```

```
# Resultado dos componentes principais
```

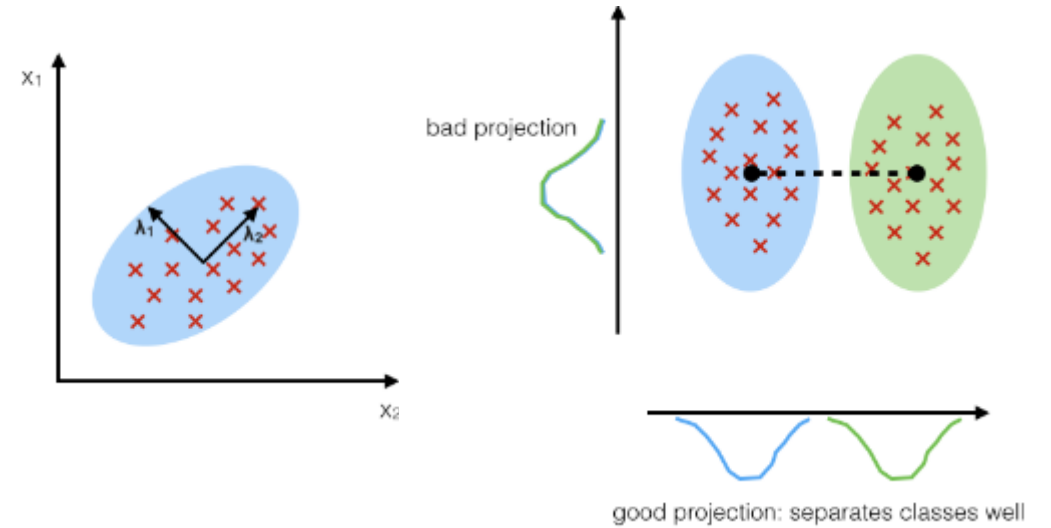
```
print(X_kpca)
```



LDA

PCA procura maximizar a variância dos dados

LDA procura maximizar a separação entre classes



```
# pip install scikit-learn
```

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import numpy as np
```

```
# Dados de exemplo
```

```
X = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0], [2.3, 2.7]])
y = np.array([0, 1, 0, 1, 0, 1]) # Classes correspondentes
```

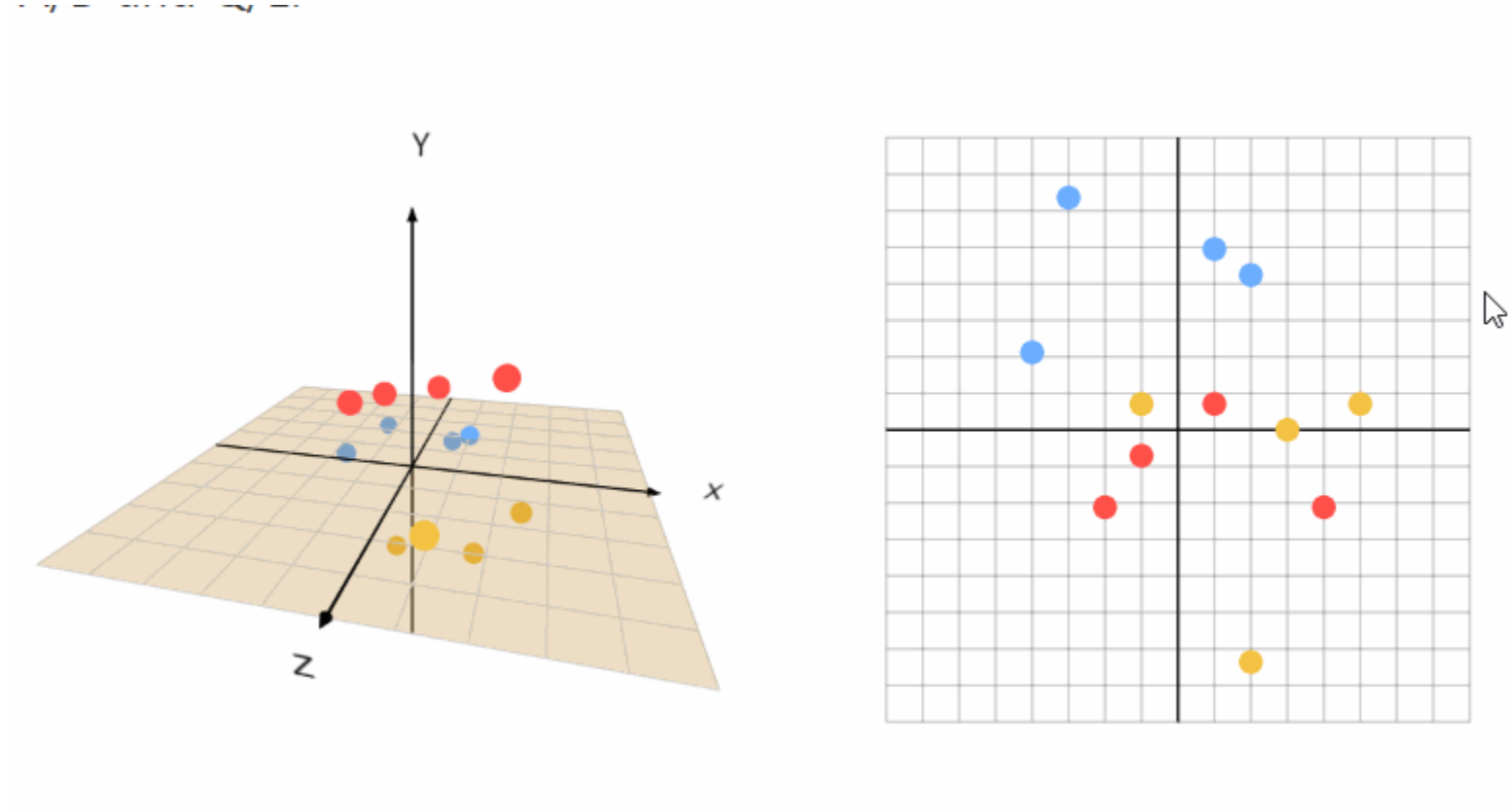
```
# Criar e ajustar o modelo LDA
```

```
lda = LinearDiscriminantAnalysis(n_components=1)
X_lda = lda.fit_transform(X, y)
```

```
# Resultado da projeção
```

```
print(X_lda)
```

LDA em 3D



- ◆ Lab 07 - Tipos de aprendizagem - aprendizagem não supervisionada
 - 7.2. - Redução de Dimensionalidade (Dimensionality Reduction) — Visualizar e Simplificar Dados Complexos